

SAP S/4HANA Integration with Keboola

Integration Guide

Setting up, running and troubleshooting the three integration flows and the Keboola adapter on SAP Integration Suite.

Contents

Overview	3
1. Before you start	3
How Keboola signs in to your tenant	3
Which topology	4
Setting up, in order	4
2. Connecting to SAP S/4HANA	5
Authentication	6
Topologies	6
Behind an SAP Cloud Connector: what to set up, and what to check first	7
3. Which services your user can reach	8
4. The pull connector	10
Retry	11
5. The delivery flow	12
Set the schedule before you deploy	13
What is retried, and what is not	14
More than one table	15
Full loads	16
Incremental loading	16
Set a primary key	16
What the flow does about a missing table	17
6. When something goes wrong	17
The dead-letter queue	19
Editions without JMS	20
7. Things worth knowing before they surprise you	20
8. The Keboola adapter	21
Settings	21
What your flow gets back	21
When it fails	22
Size	22
Appendix A. Signing in to SAP S/4HANA, step by step	22
OAuth 2.0 Client Credentials	23
Client Certificate	23
Basic	23
Switching the method	24
When the sign-in does not work	24

Overview

This package connects an SAP S/4HANA system to the Keboola data platform. It contains three integration flows:

Flow	What it does	When you use it
Query Available Services from SAP S4HANA	Lists the business data services your SAP user is allowed to reach	Once per connection, while setting it up — it fills the service list in Keboola
Query Business Data from SAP S4HANA to Keboola	Answers a request from Keboola by reading SAP and passing the answer back	Whenever Keboola asks for data
Deliver Business Data from SAP S4HANA to Keboola	Reads SAP on a schedule and writes the result into a Keboola table itself	When SAP should push, rather than Keboola pull

The first two are the **pull** topology: Keboola asks, SAP answers. The third is the **push** topology: nothing outside your SAP tenant needs to be reachable, and the delivery survives a Keboola outage because it is queued.

Everything that differs between one installation and the next is on the Configure screen of each flow. Nothing in this guide asks you to open the flow editor except where it says so explicitly, and each of those places says why.

The package also carries the **Keboola adapter**, a receiver adapter for integration flows you build yourself. None of the three flows needs it; §8 describes it.

1. Before you start

You need:

1. **An SAP Integration Suite tenant** with Cloud Integration enabled. The delivery flow also needs JMS message queues; editions without them can still use the two pull flows, and §6 describes what changes.
2. **A communication user on SAP S/4HANA** with a communication arrangement for each API you intend to read. §3 lists the arrangements for the common APIs.
3. **A Keboola project**, and — for the delivery flow — a Storage token with write access to the target bucket.

How Keboola signs in to your tenant

Keboola calls the flows with the client id and client secret of a service key. You create it in the BTP cockpit: a service instance of **SAP Process Integration Runtime**, plan `integration-flow`, created with the role the flows ask for, and a service key on that instance. The flows ship asking for `ESBMessaging.send`, a role SAP predefines in every tenant, so an instance created with `{"roles": ["ESBMessaging.send"]}` works with the package as delivered.

To let only Keboola in, use a role of your own. Every caller that holds `ESBMessaging.send` can call every flow in your tenant that asks for it. A role of your own narrows that to the callers you gave it to, and SAP's design guidelines recommend it:

1. **Monitor** → **Manage Security** → **User Roles** → **Add**. Name the role, for example `KeboolaIntegration.Send`. Letters, digits, underscore and period are allowed.
2. Create the service instance with that role — `{"roles": ["KeboolaIntegration.Send"]}` — and a service key on it. Give Keboola that key's client id and client secret.
3. In each flow you use, select the HTTPS sender (the arrow coming from *Keboola*), set **User Role** to your role, save and deploy. This is one of the few edits made in the flow editor.

Which role guards your tenant is your decision, which is why the package does not make it for you.

Reads need no token, writes do. The two pull flows have the platform's protection against cross-site request forgery switched on. Keboola only reads, and a read (`GET` , `HEAD`) is never checked, so nothing has to be set up for it. A `POST` , `PUT` or `DELETE` that carries no token is refused by the platform with `403` before the flow runs.

Which topology

	Pull	Push
Who starts the transfer	Keboola	Your SAP tenant, on a schedule
Needs your tenant reachable from Keboola	yes	no
Survives Keboola being briefly unavailable	the job fails and Keboola retries	yes — the run waits in a queue
Row-level control from Keboola's side	yes, per job	no, the schedule decides

They are not exclusive. Most installations use the catalogue and pull flows to explore and develop, and the delivery flow for the tables that matter in production.

Setting up, in order

1. **The service key for Keboola**, as described above.
2. **The security material for SAP S/4HANA**. Name it `KEB00LA_S4` and there is nothing to enter in `S4_CREDENTIAL_ALIAS`, which ships with that name. Its type follows the sign-in method (§2, Appendix A).
3. **For the delivery flow, the security material for Keboola:** *User Credentials* named `KEB00LA_STORAGE`, with any user name and the Storage token as the password.
4. **Configure** each flow you use. The fields that name your systems ship empty.
5. **In the flow editor, where needed:** the sign-in method (§2), a role of your own (above) and the schedule of the delivery flow (§5).
6. **Deploy.**

A flow that is deployed before it is configured says so. Its first call answers `500 CONFIG_ERROR` and names the setting, for example `S4_HOSTNAME is not configured on this integration flow.`

2. Connecting to SAP S/4HANA

All three flows share the same connection settings.

Parameter	Meaning
S4_HOSTNAME	Base address of the S/4HANA system, including the protocol, for example <code>https://my123456-api.s4hana.cloud.sap</code> . Must start with https:// — except behind an SAP Cloud Connector, where it is the virtual host written with <code>http://</code> (below)
S4_CREDENTIAL_ALIAS	Name of the security material that holds the sign-in details. Ships as <code>KEB00LA_S4</code>
HTTP_TIMEOUT_MS	How long one call may take, in milliseconds. Ships as <code>50000</code> on the pull connector, <code>20000</code> on the catalogue and <code>120000</code> on the delivery flow
S4_PROXY_TYPE	default for a system on the internet, <code>sapcc</code> for one behind an SAP Cloud Connector
S4_CC_LOCATION_ID	Cloud Connector location, only if you run more than one
S4_SAP_CLIENT	Client number on a multi-client on-premise system. Leave empty on S/4HANA Cloud

The `https://` rule has one exception, and it is SAP's. With `S4_PROXY_TYPE=sapcc` the address is the *virtual host* you defined in the Cloud Connector, and SAP requires it to be written with `http://` — for example `http://s4hana.virtual:44300`. The platform answers an `https://` address on that route with `405 HTTPS proxying is not supported. Use HTTP instead`. Nothing travels unencrypted: the tenant talks to the Cloud Connector through its encrypted tunnel, and the protocol of the last hop to SAP is the one you choose in the Cloud Connector's system mapping. With any other proxy type the rule stands: the pull connector refuses every request against an `http://` address with `500 CONFIG_ERROR` naming the parameter; the catalogue does the same, and adds `https://` to a bare host name; the delivery flow refuses to start a run — and its manual endpoint answers the same `500 CONFIG_ERROR` — for an `http://` value in `S4_HOSTNAME`, `KEB00LA_STACK_URL` or `KEB00LA_IMPORTER_URL`. The alternative would be sending your SAP credential, your Storage token and every row of business data across the network in clear text.

Authentication

The shipped setting is **OAuth 2.0 Client Credentials**. To use a different method you change one dropdown on the SAP S/4HANA receiver and point it at your security material. The authentication *method* is the one connection setting that cannot be externalized — the platform does not expose it as a parameter — so it is a single edit in the flow editor followed by a redeploy.

Your system	Use	Why
On-premise or private edition, reachable directly	OAuth 2.0 Client Credentials (the shipped setting)	Machine-to-machine grant issued by the ABAP OAuth server
On-premise behind an SAP Cloud Connector (S4_PROXY_TYPE=sapcc)	Basic	SAP's HTTP adapter does not allow a client certificate on that route, and a token request to an on-premise address fails there. The credential still never leaves the encrypted tunnel. Applies to all three flows
S/4HANA Cloud, public edition	Client Certificate , or Basic	Cloud inbound OAuth is usually authorization-code, which needs a browser and a person — neither exists here. Switch the receiver's dropdown and, for a certificate, name the key pair in <i>Private Key Alias</i>
Either, to get started	Basic	Works everywhere over TLS; SAP's design guidelines flag it as not the strongest choice

The catalogue flow signs in in two ways, depending on the route. On a directly reachable system it reads SAP from a script, with the user name and password of the credential named in `S4_CREDENTIAL_ALIAS` — *User Credentials*, or an OAuth2 Client Credentials artifact whose client id and secret the ABAP system also accepts as user and password. Behind a Cloud Connector (`S4_PROXY_TYPE=sapcc`) it reads through its two receivers, `HTTP_V2` and `HTTP_V4`, which ship with OAuth 2.0 Client Credentials like the other two flows: the OAuth client then needs the scopes of the two Gateway catalogue services (§3), or you switch the dropdown on both receivers. SAP's adapter does not allow a client certificate behind a Cloud Connector, and an OAuth token request to an on-premise address fails there, so Basic is the usual choice on that route.

When the security material does not fit. A material that is missing, or of another type than the sign-in method needs, is answered by the two pull flows with `500 CONFIG_ERROR` and a sentence that names `S4_CREDENTIAL_ALIAS` . In the delivery flow the same fault sends the run to `RETRY` , where it waits until the material is corrected (§5).

Appendix A has the steps for each method, click by click.

Topologies

- **S/4HANA Cloud, public edition.** Reachable on the internet; set `S4_PROXY_TYPE=default` . Note that the Gateway service catalogue is not available on this edition at all — see §3.
- **On-premise or private edition, exposed.** Same as above, pointing at your gateway host.
- **On-premise behind an SAP Cloud Connector.** Set `S4_PROXY_TYPE=sapcc` , `S4_HOSTNAME` to `http://` plus the virtual host and port, and `S4_CC_LOCATION_ID` if you run more than one Cloud Connector. No flow edit is needed beyond the sign-in method above.

Behind an SAP Cloud Connector: what to set up, and what to check first

All three flows were tested through an SAP Cloud Connector, version 2.17.2, and behave as they do over the internet: the same reads, the same catalogue and the same delivery.

In the Cloud Connector, under *Cloud To On-Premise* › *Access Control*:

Setting	Value
Back-end Type	ABAP System
Protocol	the one your SAP port speaks — HTTPS for an HTTPS port
Internal Host and Port	your SAP system, by the name its certificate is issued to
Virtual Host and Port	a name of your choice; this is what goes into S4_HOSTNAME
Principal Type	None
Resource	/sap/opu/odata, <i>Path And All Sub-Paths</i> — it covers /sap/opu/odata4 as well, the rule is "starts with"

Three things look fine and are not. Check them in this order:

1. **The mapping says *Not Reachable at once*.** The protocol does not match the port: plain HTTP against an HTTPS port gets no answer.
2. **The mapping says *Not Reachable after a few seconds*,** and *Configuration* › *On Premise* › *Backend Trust Store* reads "No trusted backends are configured". The Cloud Connector trusts no system over HTTPS until a certificate is on its allowlist. Add the certificate of the authority that signed your SAP system's certificate, or that certificate itself.
3. **The Cloud Connector says *Connected*, and every call answers *503 Timed out waiting for tunnel to open after ten seconds*.** The Cloud Connector's Java does not know the authority that signs the tunnel host of your tenant's landscape. Pairing goes to one SAP host, the data tunnel to another (`connectivitytunnel.cf.<landscape>.hana.ondemand.com`), and on the newer landscapes that one is signed under *DigiCert TLS RSA4096 Root G5*, which a Java from before 2021 does not carry. The Cloud Connector's log (`log/scc_core.trc`) then says `unable to find valid certification path to requested target` . Update the Java the Cloud Connector runs on, or add that root to its trust list (`cacerts`), and restart the Cloud Connector.

SAP's own test tells you whether the tenant reaches the Cloud Connector, without any flow: **Monitor** › **Manage Security** › **Connectivity Tests** › **Cloud Connector**, with your Location ID or an empty one.

The addresses inside SAP's answers (`__metadata` , `links`) name the virtual host on this route. The flows do not rewrite them; §4 says what that means for the pull connector.

3. Which services your user can reach

The catalogue flow answers on `https://<your tenant>/http/keboola/catalog` and returns the services your communication user is actually allowed to read. It tries three sources in order and uses the first that answers:

1. **Communication arrangements** — the authoritative list on S/4HANA Cloud. Needs the arrangement for `SAP_COM_0A07` (communication arrangement read).
2. **The Gateway service catalogues** — available on on-premise and private-edition systems. Both the OData V2 catalogue (`/sap/opu/odata/IWFND/CATALOGSERVICE;v=2`) and the OData V4 catalogue (`/sap/opu/odata4/iwfnd/config/default/iwfnd/catalog/0002`) are read and merged, so V2 services and V4 service groups both appear.
3. **A direct probe** of the shipped list of released OData services.

On S/4HANA Cloud the Gateway catalogues answer 403 and always will: no communication scenario grants them. That is not a misconfiguration and nothing you set will change it; the arrangements source is the one that works there.

Behind an SAP Cloud Connector (`S4_PROXY_TYPE=sapcc`) the flow reads the two Gateway catalogues through the connection and does not consult communication arrangements or probe — an on-premise or private-edition system has no arrangements to read. The response header `X-Keboola-Catalog-Source` says `gateway-cc` on that path, `arrangements`, `gateway` or `probe` otherwise, and `cache(<source>, <age>s old)` when the list came from the cache, so a support person can see which source answered without opening the monitor.

On an on-premise system the Gateway catalogues list everything — on a typical S/4HANA that is ten thousand entries, most of them the backends of Fiori apps and raw CDS views. The flow answers with one of three views, chosen with `?include=`:

View	What it keeps	Typical size
interfaces (default)	Services SAP classifies as <code>WEB_API</code> , services you built yourself, anything named <code>API_*</code> , and OData V4 services from the released repository (<code>srvd_a2x</code>)	a few hundred
extended	Everything SAP does not classify as a UI service; of OData V4, the released repository and the services you built yourself	under a thousand
all	Everything the catalogues list	everything

The view is the `CATALOG_VIEW` parameter (`interfaces` unless you change it); `?include=` on the address overrides it for one call, and a value that is none of the three is ignored. The views nest, and the response header `X-Keboola-Catalog-View` says which one answered and what it hid (`interfaces;shown=129;hidden=9884;denied=2;broken=73`). The list is sorted with `API_*` services first, then alphabetically, by technical name — a service your system registered as `ZAPI_BUSINESS_PARTNER_0001` sorts as `API_BUSINESS_PARTNER`. Deprecated services stay and say so in their description. On S/4HANA Cloud the views are identical: the arrangements list is curated already.

An on-premise catalogue lists what exists, not what your user may read. So the flow checks: every service in the view is asked for one record with the communication user, in parallel. Services that answer *not authorized* leave the list (the `all` view keeps them, marked `(user not authorized)`), services that answer a

server error other than 501 leave it too (marked (service error HTTP 500) in all — on a test system that is common), and the few that answer something the check cannot judge stay, marked (not verified) . The check runs once per cache period and takes 10–20 seconds on a typical system; set `CATALOG_CHECK_AUTHORIZATION=false` to skip it, or add `?filter=none` for one call. It is not possible through an SAP Cloud Connector, where the list is shown unchecked.

Every answer carries a run identifier — the platform's own message id — as `messageId` in error bodies and as the `RunId` property of the run in the message monitor; add `?debug=1` to get the diagnostics inside the answer under `d.diagnostics` (the component reads `d.results` only).

OData V2 and V4 services are listed together, each row's description ending in (OData V2) or (OData V4) , because the component's dropdown shows *Title - Description* and its *OData Version* field offers v2 and v4 by hand next to the recommended auto . The component does not tell the catalogue which one the user picked — its `loadServices` call carries `Authorization` and `Accept` and nothing else — and with `auto` it reads the chosen service's `$metadata` and finds the version itself, so the list is never filtered by version on its own. `?odata=v2` or `?odata=v4` keeps one kind, for a support call or for a component version that passes the choice along.

On a multi-client on-premise system set `S4_SAP_CLIENT` ; it is sent as `sap-client` with every call. Leave it empty on S/4HANA Cloud.

The answer is cached for `CATALOG_CACHE_TTL_SECONDS` (an hour by default) per system and user; `?nocache=1` forces a fresh read, and the value `0` switches the cache off. An **empty** list is never cached and is reported with the status `CATALOG_EMPTY` — it almost always means the communication user is not yet assigned to an arrangement. The cache is not used behind a Cloud Connector, where a read is two calls to the system's own catalogues.

A list that could not be read is an error, not an empty list. When SAP refuses the sign-in, cannot be reached or answers the catalogue requests with a server error, the flow answers 502 `UPSTREAM_ERROR` at once and says which of the three it was. It says so too when `S4_HOSTNAME` names a sign-in page or a web front end instead of the API address of the system.

Each API needs its own communication arrangement. The common ones:

Data	OData service	Communication scenario
Business partners, customers and suppliers	API_BUSINESS_PARTNER	SAP_COM_0008 Business Partner, Customer and Supplier Integration
Products	API_PRODUCT_SRV	SAP_COM_0009 Product Integration
Sales orders	API_SALES_ORDER_SRV	SAP_COM_0109 Sales Order Integration
Purchase orders	API_PURCHASEORDER_PROCESS_SRV (deprecated by SAP)	SAP_COM_0053 Purchase Order Integration
Supplier invoices	API_SUPPLIERINVOICE_PROCESS_SRV	SAP_COM_0057 Supplier Invoice Integration
G/L accounts	API_GLACCOUNTINCHARTOFACCOUNTS_SRV	SAP_COM_0241 Finance - Cloud Consolidation Data Collection Integration, or SAP_COM_0087 SAP Analytics Cloud for Planning Integration
Journal entry items	API_JOURNALENTTRYITEMBASIC_SRV	SAP_COM_0241 or SAP_COM_0087, as above
G/L account line items	API_GLACCOUNTLINEITEM	SAP_COM_0303 Finance - Accounting Analytics Integration

Customers and suppliers are read through the business partner service, so they need no arrangement of their own. SAP_COM_0002 , Finance - Posting Integration, is for posting journal entries and does not grant reading them.

The table was checked against SAP S/4HANA Cloud Public Edition, release 2608. The app *Display Communication Scenarios* in your own system is the authority for your release.

4. The pull connector

Keboola calls `https://<your tenant>/http/keboola/connector/<the full OData path>`. The path after `/connector/` is passed to SAP unchanged, so it must be the complete service path — `/sap/opu/odata/sap/API_BUSINESS_PARTNER/A_BusinessPartner`, not just the service name.

It answers `GET` and `HEAD`. Anything else is refused: by the platform with `403` when the request carries no CSRF token (§1), and by the flow with `405` and an `Allow` header when it does. The connector is a read path, and a write verb would reach SAP with an empty body and fail confusingly at the far end.

Some paths never reach the flow. The platform rejects a path that contains an encoded slash (`%2F`), an encoded backslash (`%5C`), an encoded percent sign (`%25`), a line break, a double slash or `/../` with `400`, and logs no run. An OData key that holds such a character cannot be addressed through the connector.

The flow makes one call to SAP per request and passes SAP's status and body back untouched. Five request headers travel from Keboola to SAP: `Accept`, `Accept-Language`, `Prefer`, `If-None-Match` and `If-Modified-Since`; everything else stops at the connector. Of SAP's response headers, the ones an OData client uses travel back: `Content-Type`, `Content-Length`, `Content-Encoding`, `Content-Language`, `ETag`,

Last-Modified, Cache-Control, DataServiceVersion, OData-Version, sap-message, sap-messagescount, Location and Retry-After. On a multi-client on-premise system set S4_SAP_CLIENT; the connector then adds sap-client to every query that does not already carry one.

The connector passes on any path, not only the path of an OData service. What can be read is decided in three other places: the authorizations of the user in your security material, the resources you released in the Cloud Connector, and the role that guards the endpoint in your tenant (§1).

Addresses inside SAP's answers are not rewritten. SAP's answers carry addresses of their own: of each record, of the next page when SAP pages an answer itself, and of the place a redirect leads to. They name your SAP system, not the connector, and the connector passes them on as they are. A caller that follows one leaves the connector, and Keboola's component follows both redirects and next-page addresses. So give Keboola the service path that SAP answers directly: SAP redirects the address of a service that is written without its final slash. The sign of a caller that left the connector is a Keboola job that fails with a 401 from the SAP system, or with a host that cannot be found when the system is behind an SAP Cloud Connector.

Retry

The connector does not retry; Keboola does. Keboola's SAP OData component retries an answer of 500, 502, 503 or 504 three times with exponential backoff, and both the number of retries and its timeout are settings in the component's connection options. The connector's part is to tell the truth:

What happened	What Keboola receives	Keboola retries?
SAP answered	SAP's own status and body	5xx yes; 4xx no — a 403 will still be a 403
SAP could not be reached — no DNS, refused connection, certificate failure	502 UPSTREAM_UNREACHABLE with the cause in the body	yes
SAP did not answer within HTTP_TIMEOUT_MS	504 UPSTREAM_TIMEOUT	yes
This flow is misconfigured	500 CONFIG_ERROR naming the parameter	yes, uselessly — fix the parameter
The security material named in S4_CREDENTIAL_ALIAS is missing, or of another type than the sign-in method needs	500 CONFIG_ERROR naming S4_CREDENTIAL_ALIAS	yes, uselessly — correct the security material

Keep HTTP_TIMEOUT_MS (shipped 50000) below the component's own timeout — 60 seconds unless changed — so that a slow SAP system produces a 504 Keboola can retry rather than a dropped connection. There is one retry layer, so one read costs at most four calls to SAP under the component's defaults, every one of them a separately billed message.

Every error body carries messageId: the run's Message GUID in **Monitor** → **Message Processing Logs**, so a failed Keboola job is matched to its run without any header from Keboola.

5. The delivery flow

A run is started by the schedule or by a POST to `https://<your tenant>/http/keboola/deliver` (any other verb answers 405), put on a queue, and carried out by a worker that reads SAP page by page and writes each page into Keboola.

The queue is the point. Because a run is a queued message, a failure that may pass — a server error, a dropped line, a timeout — puts the run back on the queue and the platform retries it with exponential backoff, at the two intervals you configure; while it waits it is a failed message you can see. A failure that will not change — not authorized, a wrong path, a wrong table — ends the run with a status you can filter on, and is not retried: the next scheduled run is the retry, once the cause is fixed. Same rule as the pull connector, where Keboola retries 5xx and never a 4xx (§4).

Parameter	Default	Meaning
SERVICE_PATH	/sap/opu/odata/sap/API_BUSINESS_PARTNER	Path of the service to read, without the host
ENTITY_SET	A_BusinessPartner	The collection to read from that service. A view with parameters is written whole, as in <code>Name(P_x=...)/Results</code>
ODATA_SELECT	empty	Fields to read, separated by commas; empty or * reads every field. Naming them fixes the column order and stops a later SAP release adding columns silently. Must include the columns of PRIMARY_KEY. A structured field cannot be named
ODATA_FILTER	empty	Extra condition, in SAP's query language
LOAD_MODE	full	full replaces the table's contents on every run; incremental updates it by primary key
DELTA_FIELD	LastChangeDate,CreationDate	Field that says when a record changed; several fields are separated by commas
DELTA_FIELD_TYPE	datetime	How those fields are typed: datetime, datetimeoffset or date
DELTA_PRECISION	day	day if SAP records only the date of a change, second if it records the time, in universal time
PAGE_SIZE	1000	Records per request, 1 to 100000. When SAP hands a request out in smaller slices, the flow follows them to the end

Parameter	Default	Meaning
S4_SAP_CLIENT	empty	Client number on a multi-client on-premise system; sent as sap-client
HTTP_TIMEOUT_MS	120000	Longest one call may take, a page from SAP or a call to Keboola, from 1000 to 3600000; longer counts as a passing failure and the run is retried
KEB00LA_STACK_URL	—	Your Keboola region address, for example https://connection.eu-central-1.keboola.com
KEB00LA_TABLE_ID	—	The table's identifier as Keboola shows it, for example in.c-sap.business_partner. The bucket must exist; the flow creates the table
KEB00LA_CREDENTIAL_ALIAS	KEB00LA_STORAGE	Security material holding the Storage token: <i>User Credentials</i> , any user name, the token as the password
KEB00LA_IMPORTER_URL	empty	Only to override the address looked up from the region
PRIMARY_KEY	BusinessPartner	Columns identifying a record uniquely, separated by commas; required for incremental loading
DELIVERY_QUEUE_NAME	KeboolaDelivery	Queue holding runs. Every copy of the flow needs a queue of its own
DELIVERY_RETRY_INTERVAL_MIN	3	Wait before the first retry of a run that failed for a passing reason; doubles each time
DELIVERY_MAX_RETRY_INTERVAL_MIN	60	The longest wait between retries
AIR_KEY, AIR_HEADER_NAME	empty	SAP's integration key, once issued (§7)

The shipped values read business partners. `SERVICE_PATH`, `ENTITY_SET`, `DELTA_FIELD` and `PRIMARY_KEY` ship as one example that works together. When you read another collection, change all four: the shipped key and change fields belong to business partners.

Set the schedule before you deploy

The flow ships with its schedule set to **Run Once**, which means **every deploy starts one delivery** with whatever is configured at that moment. Set your real schedule first: open the flow, select the **On Schedule** start event, and use the Scheduler tab. The package does not offer the schedule on the Configure screen, so this is a step in the flow editor.

To run a delivery without waiting for the schedule, `POST` to `/http/keboola/deliver`. It answers `202` with the envelope of the run it started, `messageId` first. That `messageId` is made by the flow, because the run itself is processed later from the queue: it is the run's *Application Message ID* in **Monitor** → **Message Search** and the log field `KeboolaMessageId`. A refused start answers `500` with `CONFIG_ERROR` and the setting named, or `405` for a wrong verb, and the body's `messageId` is then the run's id in Message Processing Logs. The endpoint asks for no CSRF token.

The settings are checked before anything is queued. A start is refused when `SERVICE_PATH` or `ENTITY_SET` is empty or holds a question mark or a blank, when `HTTP_TIMEOUT_MS` is outside its range, when `ODATA_SELECT` leaves out a column of `PRIMARY_KEY`, when incremental loading has no `PRIMARY_KEY`, when an address does not start with `https://` (§2), when a required setting is empty, when `LOAD_MODE`, `PAGE_SIZE`, `DELTA_FIELD_TYPE` or `DELTA_PRECISION` is outside its list or range, and, for incremental loading, when `DELTA_FIELD` is empty or not a comma-separated list of field names.

On the schedule there is nobody to answer. A start that is refused there is a run with the status `FAILED` and the same sentence in its error text. Every deploy also leaves one entry with the status `DISCARDED` and no sender; it belongs to the timer and is not a failure.

What is retried, and what is not

What happened	The run	Retried?
SAP answered 5xx, 408 or 429, or did not answer within <code>HTTP_TIMEOUT_MS</code> , or the line dropped	goes back on the queue, log entry <code>RETRY</code>	yes, by the platform, at the two intervals, until it succeeds or the message expires
Keboola's service index or importer answered 5xx, or did not answer	same	same
SAP answered a 4xx (403 no arrangement, 404 wrong path, 400 bad filter)	ends <code>UPSTREAM_FAILED</code> , SAP's text attached, watermark not moved	no — fix the cause; the next run reads the same window again
SAP answered with something that is not a list of records: a web page, XML, JSON of another shape, or nothing	ends <code>UPSTREAM_FAILED</code>	no — <code>S4_HOSTNAME</code> probably names a sign-in page or a web front end, not the API address of the system
SAP returned the same records for two requests in a row: the service does not apply <code>\$skip</code> , or <code>PRIMARY_KEY</code> does not identify one record (records that carry no address of their own, as in OData V4, are compared by their <code>PRIMARY_KEY</code> values)	ends <code>UPSTREAM_FAILED</code>	no — check <code>PRIMARY_KEY</code> ; if it is right, set <code>PAGE_SIZE</code> above the number of records, or read a service that pages
Keboola answered 401/403/400 — to the service index (the token), to the table check or to the import — or the table could not be created for a reason that will not change	ends <code>KEB00LA_FAILED</code>	no
Keboola answered with a redirect	ends <code>KEB00LA_FAILED</code> ; the Storage token is never sent after a redirect	no — correct <code>KEB00LA_STACK_URL</code>

What happened	The run	Retried?
The table exists in Keboola with another primary key than PRIMARY_KEY	ends KEB00LA_FAILED with both keys named; nothing is imported	no — set the same key in both places
A setting is missing or wrong	the start is refused: 500 CONFIG_ERROR on the manual endpoint, a FAILED run on the schedule; inside a run, ends CONFIG_ERROR	no
The run needs more pages than the flow reads in one run	ends CONFIG_ERROR, watermark not moved	no — raise PAGE_SIZE or narrow ODATA_FILTER
The sign-in to SAP could not be prepared: the security material named in S4_CREDENTIAL_ALIAS is missing, or of another type than the sign-in method needs	goes back on the queue, log entry RETRY	yes, until you correct the security material; the run then completes at its next retry

One repeat is not a failure. When SAP answered a request with more records than were asked for and the next request brings the first records again, SAP had handed out everything already: the run ends `DELIVERED` without importing the repeat, and its log says `page <n>`: the records of the first request again, not delivered: SAP S/4HANA had already handed out every record.

Every retry is a run of the worker and at least one billed call to SAP, which is why a fault that would answer the same way every hour is not retried. The last row is the one exception: the platform reports that fault before the flow can judge it. Correct the security material, or delete the waiting message under **Monitor** → **Manage Message Queues**.

One boundary: a single run must finish within **four hours**, the time the queue keeps a run locked to the worker that took it (the sender's Lock Timeout, shipped at 240 minutes). Size `PAGE_SIZE` and `ODATA_FILTER` so that it does; a load that needs longer is split over two copies of the flow, each with a filter and a table of its own (below).

A second boundary: **one run per table at a time**. Two runs writing the same table at once — a schedule and a manual `POST`, or a deploy's own Run Once delivery and a `POST` right after it — wait for each other at Keboola's importer, and each page takes longer. The shipped `HTTP_TIMEOUT_MS` of `120000` allows for that. With a lower value both runs go to `RETRY` and their retries fall due together again. Start the second run when the first shows `DELIVERED`; the flow's own concurrency setting does not keep two runs apart.

More than one table

One flow delivers one collection into one table. For another table, or to split a large load, copy the flow and give the copy three things of its own:

1. **An address**, set on the HTTPS sender in the flow editor, for example `/keboola/deliver/products`. Two flows cannot share an address.
2. **A queue**, in `DELIVERY_QUEUE_NAME`. Copies that share a queue carry out each other's runs.
3. **A table**, in `KEB00LA_TABLE_ID`.

Two deliveries must not write into the same table. A full load of one replaces what the other delivered, and the point reached by incremental loading is kept per table, so each would skip what the other has yet to read.

Full loads

A full load replaces the table: the first page that carries records is imported as a full import, which clears what was there, and every later page is appended. That is page 1, unless SAP begins with an empty answer and a link to more. A full load is therefore **not atomic** — between the first and the last page a reader sees a partial table. If something reads the table continuously, use incremental loading with a primary key instead, where every page is an update.

A full load that reads no records leaves the table as it was. One rare case: if SAP answers a retried run differently from its first attempt, with records where the first attempt found none, the full load can add to the table instead of replacing it. The next full load puts it right.

Incremental loading

With `LOAD_MODE=incremental`, each run reads the records whose `DELTA_FIELD` lies between the start of the previous **successful** run and the start of this one — both ends inclusive, at the precision you set. At the precision `day` the period opens one day earlier still. Most SAP change fields carry only a *date*, and it is the date of the SAP system's own time zone, while the flow keeps its times in universal time: without the extra day a record changed late on the day of the previous run could be missed for good. Reading those days again costs a few updates through the primary key; skipping them would lose rows. Set `DELTA_PRECISION=second` only for a field that carries a time in universal time, and `DELTA_FIELD_TYPE` to how the field is typed in the service — the flow writes the condition accordingly for both OData versions.

Name the creation field as well. Many SAP services leave the change field empty until a record is changed for the first time, so a condition on that field alone never reads a new record. `DELTA_FIELD` therefore takes several fields, separated by commas, and a record is read when any of them lies in the period. All of them must be of the type named in `DELTA_FIELD_TYPE`.

The period each run used is in monitoring as `DeltaWindow`, and the first query as `FirstPageQuery`. The platform keeps the first 200 characters of each.

The point reached is written only after every page has landed, so a run that fails half way re-reads its whole window next time rather than skipping the rows it never delivered. The first incremental run has nothing stored yet and reads everything up to its own start. The point reached is kept in the tenant's data store `KeboolaDeliveryState` under the table's identifier (**Monitor** → **Manage Stores** → **Data Stores**); delete the entry, and the next run reads everything.

Incremental loading **requires** `PRIMARY_KEY`: the flow refuses to start without one.

Set a primary key

With `PRIMARY_KEY` set, delivering the same record again updates it, and the records are read in key order so that consecutive pages are consecutive slices of one ordering. Without one, a redelivered record is added again. Set it whenever you can — and always for incremental loading.

The key is given to the table when the flow creates it. A table that exists already keeps the key it has in Keboola; when `PRIMARY_KEY` is set and differs from it, the run ends `KEB00LA_FAILED` and names both.

The flow also refuses to import the same page of the same run twice, so a redelivered run does not re-import the pages that already landed. One case escapes it: a page that Keboola finished importing only after the flow had stopped waiting for the answer counts as not delivered, and the retry imports it again. With a primary key its records replace themselves. Without one they are in the table twice until the next full load.

The primary key is the second line of defence, and the one that also covers a record changing between two different runs. One residual case remains, and it is inherent to offset paging: rows inserted in SAP between two attempts of the *same* run can shift the page boundaries, so a page skipped as already delivered may not hold exactly the rows it held the first time. The next run's inclusive window reads them.

What the flow does about a missing table

The service that receives the data cannot create tables. On the first page that carries records the flow asks whether the target table exists; if it does not, it creates it from the columns of the data being delivered, with `PRIMARY_KEY` as the key, and waits for the creation to finish before importing. Every column is created as text: the flow cannot invent a business meaning SAP does not declare, and a wrong type is far more expensive to undo than a conversion later. The columns stand in the order of `ODATA_SELECT`, and in alphabetical order when that is empty.

6. When something goes wrong

Every failure is visible in **Monitor** → **Message Processing Logs**, filterable by status without opening a single message.

Status	Meaning	What to do
OK	A pull request or a catalogue call was answered	—
QUEUED	A delivery run was put on the queue	—
DELIVERED	Every page of a delivery run landed	—
RETRY	A delivery run failed for a reason that may pass and is waiting on the queue	Nothing, unless it keeps happening: Monitor → Manage Message Queues shows the retry count and the next retry. A run whose sign-in to SAP could not be prepared waits there until the security material is corrected (\$5)
UPSTREAM_FAILED	SAP refused the request and said why. In a delivery run: SAP refused a page, or answered with something that is not a list of records, and the run ended without a retry	Read the fields <code>UpstreamStatus</code> , <code>UpstreamErrorCode</code> and <code>UpstreamError</code> : SAP's own code and sentence. A delivery run has no <code>UpstreamErrorCode</code> , and its <code>UpstreamError</code> is the first 200 characters of SAP's answer. For a delivery run read the attachment <code>ErrorDetails</code> as well; the next run repeats the window. Usually a missing communication arrangement
UPSTREAM_UNREACHABLE, UPSTREAM_TIMEOUT	The connection could not be made, or SAP did not answer in time	Check <code>S4_HOSTNAME</code> , the Cloud Connector, and network reachability; for a timeout, SAP's own health and <code>HTTP_TIMEOUT_MS</code> . Keboola retries these on its own

Status	Meaning	What to do
UPSTREAM_ERROR	The catalogue could not read its list: SAP refused the sign-in, could not be reached, or answered with an error. The message says which	For a refused sign-in check the user and password in the security material, and that the user is not locked in SAP; otherwise S4_HOSTNAME and the route to the system
KEB00LA_FAILED	Keboola refused the import or the table for a reason that will not change	Read the fields KeboolaStatus and KeboolaError (the first 200 characters of Keboola's answer) and the attachment ErrorDetails: token, bucket rights, table, primary key
CONFIG_ERROR	This flow is misconfigured	The message names the parameter. When it names S4_CREDENTIAL_ALIAS, the security material is missing or of another type than the sign-in method needs
METHOD_NOT_ALLOWED	The caller's request was refused	The body says why
DEGRADED_PROBE	The catalogue answered from a probe of the shipped service list, because neither the arrangements nor the Gateway catalogues could be read	On S/4HANA Cloud grant SAP_COM_0A07 to the communication user; on an on-premise system give the user read access to the two Gateway catalogue services
CATALOG_EMPTY	The catalogue found nothing	Assign the communication user to an arrangement; the empty answer is not cached
CATALOG_ERROR	An unexpected failure inside the catalogue flow	The message and the attachment ErrorDetails say what failed
DELIVERY_NOT_STARTED	A manually started delivery could not be queued	The body says why
FAILED	A scheduled delivery was refused at its start	The error text of the run names the setting (§5)
DISCARDED	Left by the timer at every deploy of the delivery flow	Nothing; it is not a failure

A 401 or 403 in a Keboola job has two possible origins. When the run exists in the monitor with the status `UPSTREAM_FAILED`, SAP refused the user of the security material. When no run exists, your tenant refused Keboola's own key or role (§1).

Behind an SAP Cloud Connector a problem with the tunnel is answered by SAP's platform, not by SAP S/4HANA and not by this package, and the pull connector passes the answer on as it is:

Answer	SAP's sentence begins	Cause
403	Access denied to resource ... on system ...	The path is not among the mapping's resources
503, within a second	There is no SAP Cloud Connector (SCC) connected to your subaccount ... and SCC location ID ...	No Cloud Connector is paired under that Location ID: check S4_CC_LOCATION_ID and the pairing
503, after about ten seconds	Timed out waiting for tunnel to open ...	The Cloud Connector is paired but cannot open the data tunnel: §2, the third check
405	HTTPS proxying is not supported. Use HTTP instead ...	S4_HOSTNAME starts with https://; on this route it is http:// plus the virtual host

The catalogue answers 502 UPSTREAM_ERROR instead and names the status of both catalogue requests in its message. The delivery flow records the status and SAP's sentence in the fields UpstreamStatus and UpstreamError: a 503 sends the run to RETRY, a 403 or 405 ends it UPSTREAM_FAILED. A run that waits in RETRY because of the tunnel is retried or deleted under **Monitor** → **Manage Message Queues** once the cause is fixed.

Searchable fields on every connector run: HttpPath, UpstreamStatus, UpstreamOutcome. When SAP refused the request, also UpstreamErrorCode and UpstreamError — SAP's own code and sentence. SAP's whole answer goes back to Keboola and is not kept in the log; what Keboola sent is in the log property IncomingHeaders, with credentials redacted. When the call could not be made at all, the attachment ErrorDetails holds the cause and the attachment IncomingHeaders the request. For deliveries: Pages, RowsDelivered, TableId, Watermark, DeltaWindow, FirstPageQuery, PageOutcome and TableState per page; on a refused page UpstreamStatus, UpstreamOutcome and UpstreamError; on a refused import KeboolaStatus and KeboolaError. These are fields, not attachments, so they can be filtered on and read by API; the platform keeps the first 200 characters of a field. Runs also carry SAP's standard search identifiers, so **Monitor** → **Message Search** finds them by sender, receiver or message type — and a connector run by the path that was read, which is its *Application Message ID*.

A failed pull request is not stored anywhere else: it is answered to Keboola in the same second, the Keboola job fails visibly with the connector's error body, and Keboola's own notifications apply. The run stays in Message Processing Logs for the tenant's retention period, findable by the messageId in that body.

The dead-letter queue

A delivery run that fails for a reason that may pass is retried by the platform with growing intervals — DELIVERY_RETRY_INTERVAL_MIN doubling up to DELIVERY_MAX_RETRY_INTERVAL_MIN — without a retry count: it keeps being retried until it succeeds or the queued message expires, 90 days after it was queued. While it waits it is a failed message in **Monitor** → **Manage Message Queues**, with its retry count and next retry time, and that is where you retry or delete it after fixing the cause. The platform's own dead-letter queue takes only a message whose processing crashed the runtime node, after two attempts. A run that fails for a reason that will not change is not queued for retry at all: it ends with UPSTREAM_FAILED, KEB00LA_FAILED or CONFIG_ERROR in Message Processing Logs, and the next scheduled run is the retry.

Editions without JMS

The delivery flow needs message queues and cannot be used on an edition without them. On such an edition:

- Do not deploy the delivery flow; deploying it fails for want of a queue.
- The two pull flows work unchanged, and Keboola's own scheduling replaces the delivery flow's.
- The supported fallback for error handling is the pull topology itself: every failed read is answered to Keboola synchronously with a status and a body naming the run, retried by Keboola with exponential backoff where retrying can help, surfaced in Keboola's job log and notifications, and filterable in this tenant's Message Processing Logs by custom status. Nothing waits in a queue, so nothing can be lost from one.

7. Things worth knowing before they surprise you

Retries are billed. Every call to SAP is a separately billed message, and a retry is another one. The pull connector makes one call per request and leaves retrying to Keboola, whose three retries are the ceiling; the catalogue's defaults are deliberately modest. The delivery flow's queue is not metered, but every retried run calls SAP again, which is why a fault that will not change ends the run instead of being retried.

Data residency. The data moves from your SAP system to your Integration Suite tenant to your Keboola project. Those three can be in different regions; choose the Keboola region and the tenant region deliberately.

Date formats. Older SAP services return dates as `/Date(1777939200000)/` and times as `PT19H02M07S`. The delivery flow rewrites both into the standard form, always. It writes numbers in plain notation, and it does not deliver the control information of OData V4 (`@odata.etag` and the like) as columns. The pull connector passes SAP's answer through untouched — it is a proxy, and rewriting a payload in transit would make it something else.

The service list is a snapshot. The catalogue flow carries a list of released OData services taken from SAP's published catalogue. It drifts as SAP releases new ones; the arrangements source does not, which is why it is tried first.

Later versions of a service. SAP publishes a few OData V2 services in a second version; the shipped list holds twelve of them. It names them under the address of the first version, and Keboola's component leaves a version marker (`;v=0002`) out of an address, so it is the first version that is read.

Paging depends on SAP's answers. The delivery flow asks for `PAGE_SIZE` records at a time and follows SAP's links to the rest. An answer with fewer records and no link is taken as the end, so a service that cuts its answers short without a link loses the rest unnoticed; no client can see that. A service that ignores `$skip` answers every request with the same records; the flow notices on the second request and ends the run `UPSTREAM_FAILED ($5)`, unless SAP had handed out everything in one answer. One that ignores `$skip` and holds exactly `PAGE_SIZE` records ends that way too, because nobody can tell whether more exist. Records are told apart by their address, or without one by their `PRIMARY_KEY` values; with neither, such a service is read until the flow's page limit. For an OData V4 service, set `PRIMARY_KEY` to its key.

The integration key. If SAP issues this package a key for its Application Interface Register, put it in `AIR_KEY` and the header name SAP gives you in `AIR_HEADER_NAME`. Nothing is sent until both are filled in. The key goes to SAP only; it is not returned to the caller.

8. The Keboola adapter

The adapter is for integration flows of your own. A flow that ends with it hands over the message body, comma separated values with a header line, and the adapter writes those rows into one Keboola Storage table and gives Keboola's answer back to the flow. It reads nothing from SAP and it does not page, queue or retry: whatever starts your flow and whatever feeds it rows is yours to build. When you want SAP data in Keboola without building anything, use the delivery flow ([\\$5](#)), which needs no adapter.

Install it before the flow that uses it. Deploy the *Keboola* integration adapter from the package's Artifacts tab first, then deploy your flow. After deploying a newer version of the adapter, deploy the flows that use it once more, so that none of them keeps working with the version it started with.

The credential. Create *User Credentials* under Monitor, Manage Security, Security Material: any user name, and the Keboola Storage token as the password. The token needs write access to the bucket of the target table. The token is read from there at every delivery and appears in no setting, log or error text.

Settings

Setting	Default	Meaning
Keboola Stack URL	—	The address of your Keboola stack, for example <code>https://connection.eu-central-1.keboola.com</code> . Must start with <code>https://</code>
Credential Name	KEB00LA_STORAGE	The name of the User Credentials above
Table	—	The full identifier of the table, for example <code>in.c-sap.business_partner</code> . The table must exist , with the columns of your header line; the adapter does not create it
Incremental Load	on	On: the rows are added. Off: the table's content is replaced by this delivery
Timeout (in ms)	60000	How long connecting and waiting for Keboola's answer may take. Must be greater than 0. Keboola answers when the import is finished; a file of a few rows typically takes 15 to 20 seconds

Every setting can be externalized.

What your flow gets back

On success the message body is Keboola's answer, for example

`{"importedRowCount":315,"totalRowCount":315,"importedColumns":["id","name"],...}`, and the adapter sets three message headers:

Header	Content
KeboolaHttpStatus	the status Keboola answered with
KeboolaTableId	the table written to
KeboolaIncremental	true or false

In a flow that answers an HTTPS caller, message headers travel back as response headers; remove the three in your flow if the caller should not see them. Towards Keboola the adapter sends one header of its own, `X-StorageApi-Token`, and passes none of the message's headers on.

When it fails

A failed delivery raises an exception in your flow, so an exception subprocess or the sender's retry decides what happens next. The text names the cause:

Text	What to do
... failed with status 401: Keboola did not accept the Storage token	The token in the credential is wrong, expired or revoked
... failed with status 404: Table ... not found	Create the table, or correct <i>Table</i>
... failed with status 4xx: <Keboola's sentence>	Keboola refused the file; the sentence is Keboola's own
Could not find credential for the given alias ...	<i>Credential Name</i> names nothing that is deployed
Keboola Stack URL must start with https://...	The address would send the token unencrypted; the adapter does not call it
Could not reach the Keboola service list at ... / ... answered with status ...	The stack address is wrong or Keboola does not answer. Nothing was sent; trying again is safe
The Keboola service list ... names no importer	The address is not a Keboola stack's connection address
Could not reach the Keboola importer at ...: Read timed out	Keboola did not answer within the timeout. The import may still have happened: check the table before sending the same rows again, or deliver with <i>Incremental Load</i> on into a table with a primary key, where a repeated row replaces itself
... the address redirects to ...; redirects are not followed	Something between your tenant and Keboola answers in Keboola's place. The adapter sends the token and the rows to the configured stack only

Size

The adapter sends one message as one file and holds that message in memory while it does. Keep a delivery to tens of megabytes and split larger data into several messages, as the delivery flow does with its pages.

Appendix A. Signing in to SAP S/4HANA, step by step

This appendix is about one connection: from the flows in your tenant to your SAP S/4HANA system. How Keboola signs in to your tenant is a different connection, described in §1.

Which method suits which system is the table in §2. The flows ship with OAuth 2.0 Client Credentials, so for that method there is nothing to change in the flow editor.

OAuth 2.0 Client Credentials

1. **Monitor** → **Manage Security** → **Security Material** → **Create** → **OAuth2 Client Credentials**.

2. Fill in:

Field	Value
Token Service URL	The token address of your SAP system. On-premise and private edition: <code>https://<host>/sap/bc/sec/oauth2/token?sap-client=<client></code>
Client ID, Client Secret	The OAuth client registered on the SAP system
Client Authentication	Send as request header, unless your SAP system expects it in the body

3. Deploy the security material.

4. On the Configure screen of each flow you use, set `S4_CREDENTIAL_ALIAS` to the name of that security material.

Client Certificate

1. **Monitor** → **Manage Security** → **Keystore**. Add a key pair, or create one, and note its alias.

2. On the SAP system, assign the public certificate of that key pair to your communication user.

3. Switch the method to *Client Certificate* as described below, and enter the alias of the key pair in *Private Key Alias*.

The catalogue flow does not use the certificate. On a directly reachable system it signs in with the user name and password of the credential named in `S4_CREDENTIAL_ALIAS`, so that flow needs *User Credentials* as described under Basic below.

This method cannot be used behind an SAP Cloud Connector (§2).

Basic

1. **Monitor** → **Manage Security** → **Security Material** → **Create** → **User Credentials**. Enter the communication user of the SAP system and its password.

2. Deploy the security material.

3. On the Configure screen of each flow you use, set `S4_CREDENTIAL_ALIAS` to the name of that security material.

4. Switch the method to *Basic* as described below.

Switching the method

The method is set on the connection that leads to SAP S/4HANA, once in each flow you use.

Flow	Connection to change
Query Business Data from SAP S4HANA to Keboola	HTTP
Deliver Business Data from SAP S4HANA to Keboola	HTTP
Query Available Services from SAP S4HANA	HTTP_V2 and HTTP_V4; they are used behind an SAP Cloud Connector only (§2)

1. **Design** → open the package, open the flow, choose **Edit**.
2. Select the arrow that leads to the participant *SAP_S4HANA*.
3. On the **Connection** tab, set **Authentication** to your method.
4. **Save**, then **Deploy**.

Everything else that differs between installations, the address, the name of the credential and the timeout, is on the Configure screen and needs no edit of the flow.

When the sign-in does not work

What you see	Cause
500 CONFIG_ERROR that names S4_CREDENTIAL_ALIAS, from the pull connector or the catalogue	The security material is missing, or of another type than the method needs
A delivery run that stays in RETRY	The same fault in the delivery flow. The run completes at its next retry once the security material is corrected
401 from SAP with the status UPSTREAM_FAILED on the pull connector, 502 UPSTREAM_ERROR on the catalogue	SAP refused the user or the password, or the user is locked in SAP

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.